



Basi di Dati - Modulo Laboratorio
A.a. 2025-2026

Lezione 7: Introduzione a Python per differenze con Java

Dr. Beatrice Amico



Linguaggio Python - Introduzione

- Python è un linguaggio di programmazione ad alto livello introdotto nel 1991.
- Caratteristiche strutturali essenziali: linguaggio multiparadigma, interpretato, con tipizzazione dinamica.
- Proprietà riconosciute:
 - sintassi più semplice di molti altri linguaggi,
 - disponibilità di strutture dati ad alto livello versatili,
 - possibilità di esprimere concetti/costrutti in meno righe rispetto a quanto possibile con altri linguaggi,
 - permette di essere più produttivi rispetto ad altri linguaggi nello sviluppo di codice.



Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Tipo di programmazione	Supporta quasi tutti gli aspetti di programmazione orientata agli oggetti. Si possono scrivere programmi senza oggetti.	Supporta solo programmazione ad oggetti.
Tipo di esecuzione	Pensato per essere usato iterativamente tramite interprete (come SQL).	Programmi Java devono essere compilati esplicitamente (.class file). Class file vengono interpretati.

Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Gestione tipi di dati	DINAMICA - Una variabile è definita assegnando un valore: pippo = 42 - Una variabile può cambiare tipo in un programma: v = 2 v = 'Due' v = 'Due' + 2	STATICA - Una variabile deve essere dichiarate esplicitando il tipo prima dell'uso: int v = 2; - Una variabile NON può cambiare tipo in un programma: int v = 2; v = "Due";

Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Tipi di dati scalari	• int (unlimited precision) • bool • float (double precision) • complex	• byte (1 byte), short (2 byte), int (4 byte) • long (8 byte) • boolean • float/real • char (2 byte)
Strutture dati native	• list • tuple • range • str • set • dict	• java.util.List • ? • ? • String • java.util.Set • java.util.Map

Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Delimitatore istruzioni	Carattere di fine riga termina un'istruzione. '\ ' per spezzare un'istruzione su più righe. Indentazione delimita un blocco (usare TAB o spazi, ma non mescolare!): <pre>i = 10 nomeFunzioneLungo (a, \ b, e, f) while i > 10: i -= 1</pre>	Carattere ';' termina un'istruzione. Più istruzioni possono stare su una riga.



Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Commenti	'#' inizia un commento di una riga.	'//' inizia un commento di una riga. '/*' per iniziare e '*/' per terminare commenti anche su più righe.

Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Lista	Elenco di oggetti anche di tipi diversi. <pre>l = [1, 'uno', 2] l[0] # l elem. l[1:3] # dal II al III l[1] = 1 l = list ()</pre>	<pre>import java. util .*; List <Object> l = new ArrayList<>(); l.add (1); l.add("uno"); l.add(2); l.get(0); l.subList(1,3); l.set(1,1);</pre>



Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Tupla	Lista di valori immutabile. <pre>t = (1, 'uno ', 2) t[0] # I elem . t[1:3] # dal II al III t = tuple () t[1] = 2</pre>	

Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Range	Lista di interi immutabile. <code>range(1, 6)</code> # (1 ,2 ,3 ,4 ,5) <code>range(1, 8, 2)</code> # (1 ,3 ,5 ,7) <code>range(0, -6, -2)</code> # (0 , -2 , -4)	

Linguaggio Python - Comparazione con Java

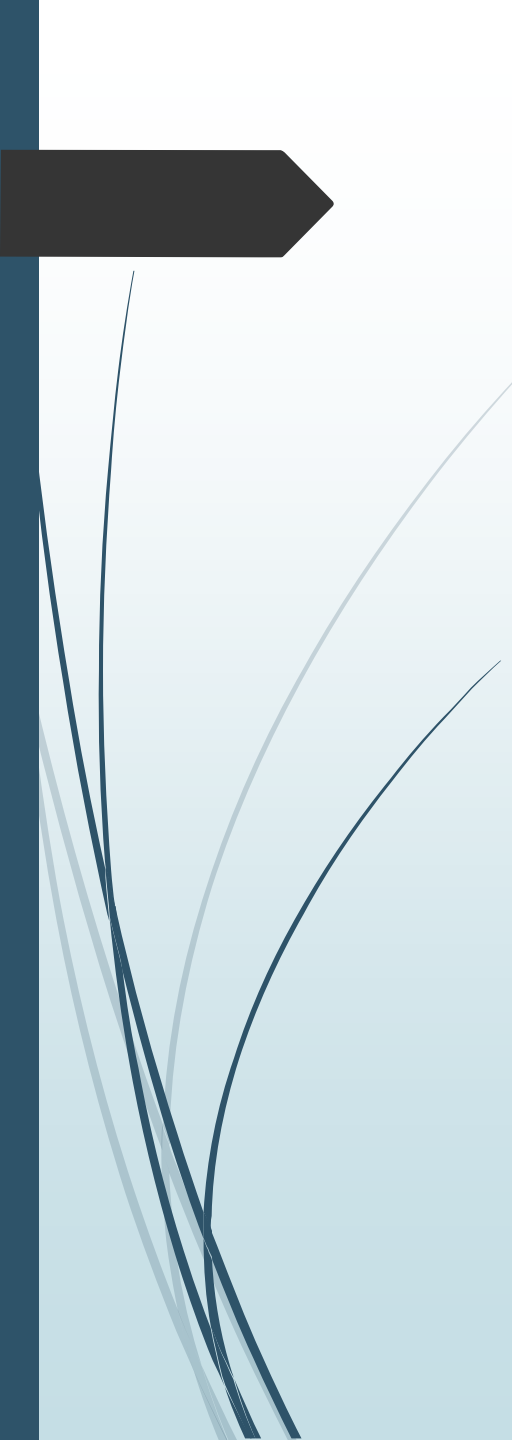
Caratteristica	Python	Java
Stringhe	Coppia di ' ' o di " " delimitano stringhe. Coppia di ''' ''' delimita stringhe su più righe: 'Sono una stringa ' " Anch 'io" ''' Io sono più lunga '''	Coppia di ' ' delimita un carattere. Coppia di " " delimita una stringa.

Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Insieme	È una lista senza elementi ripetuti e senza un ordine fissato: <pre>s = {'Bob', 3, 4} 2 in s # false 3 not in s # false</pre>	<pre>import java.util.*; Set<Object> s = new HashSet<>(); l.add("Bob"); s.add(3) ; s.add(4) ; s.contains(s); // false !s.contains (3); // false</pre>

Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Dizionario	Array associativo. Chiave solitamente è un numero/stringa: <pre>d = {'name': 'Bob ', \ 3:4} d['name'] # è Bob d[3] # è 4 'name' in d #True</pre>	La struttura dati più simile è <code>java.util.HashMap</code>.



Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Conversioni	Dato che è fortemente tipizzato, espressioni con tipi che non siano estensioni uno dell'altro non sono ammesse. Esistono molte funzioni predefinite per convertire da un tipo all'altro. Vedere http://www.tutorialspoint.com/python/python_variable_types.htm	

Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Operatori logici	not/and/or	!/&&/
Operatori di comparazione	>, <, >=, <=, ==, != si applicano anche a stringhe e altri tipi di oggetti: 'Tre' > "Due" # True [1, 2, 2] == [2, 2, 1] # False {1, 2, 3} == {3, 2, 1} # True { 'a', 'b' } < {1, 2, 3} # Sottoinsieme\ False	>, <, >=, <=, ==, != si applicano solo ai tipi primitivi. ==, != per confrontare oggetti ma...



Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Elemento vuoto	None Esempio: obj = None	null

Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Cicli	<pre>for i in range (1, 10): print(i) # i = 1, 2,..., 9 for i in range (1, 10, 2): print(i) # i=1, 3, 5, 7, 9 for letter in 'Python': print ('Letter : ' + letter) fruits = ['peach', 'apple', 'mango'] for fruit in fruits : print ('Fruit : '+ fruit)</pre>	<pre>for(int i=1; i <10; i++) System.out.println(i); for(int i=1; i <10; i +=2) System.out.println(i); for (char c : " Java".toCharArray()) System.out.println(c); String[] fruits = {"peach","apple","mango"}; for(String f : fruits) System.out.println(f);</pre>

Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Cicli While	<pre>while x > 0: ...</pre>	<pre>while(x > 0){ ... } do { ... } while(x > 0)</pre>

Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Istruzione if	<pre>if x < 0: ... </pre> <pre>if x < 0: ... else : ... </pre> <pre>if x < 0: ... elif x > 0: ... else : ... </pre>	<pre>if (x < 0) { ...; ...; } if (x < 0) ...; else ...; if (x < 0) ...; else if (x > 0) ...; else ...; C'è anche lo switch</pre>

Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Stampare su console	In Python 3.*: <code>print(<var>/ cost, ...)</code>	<code>System.out.println(...);</code> <code>System.out.print(...);</code> <code>System.out.printf(...);</code>
Leggere da console	In Python 3.*: <code>i = input("Inserire numero: ")</code> <code>i = int(i)</code> Il tipo del dato letto è sempre stringa!	// Java 7 <code>Scanner keyb = new Scanner(System.in);</code> <code>System.out.println("Enter an integer");</code> <code>int i = keyb.nextInt();</code>

Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
File I/O	<pre>f = open('workfile', 'r') # altri modi: r+, w, w+, .. for line in f: __print (line, end='') l = f.read() # legge tutto! l = f.readline() # legge una riga</pre>	<pre>File d = new File ("workfile") FileReader fr = new FileReader(f); BufferedReader in = new BufferedReader(fr); String line = null; while((line = in.readLine ()) != null) { System.out(line); } in.close (); fr.close ();</pre>

Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Definizione di funzione	<pre>def nome(parametri): corpo1 corpo2</pre> Esempio : <pre>def disc(a, b, c): return b*b - 4*a*c</pre>	Esempio : <pre>class Solver { double disc(double a, double b, double c){ return b*b - 4*a*c; } }</pre>

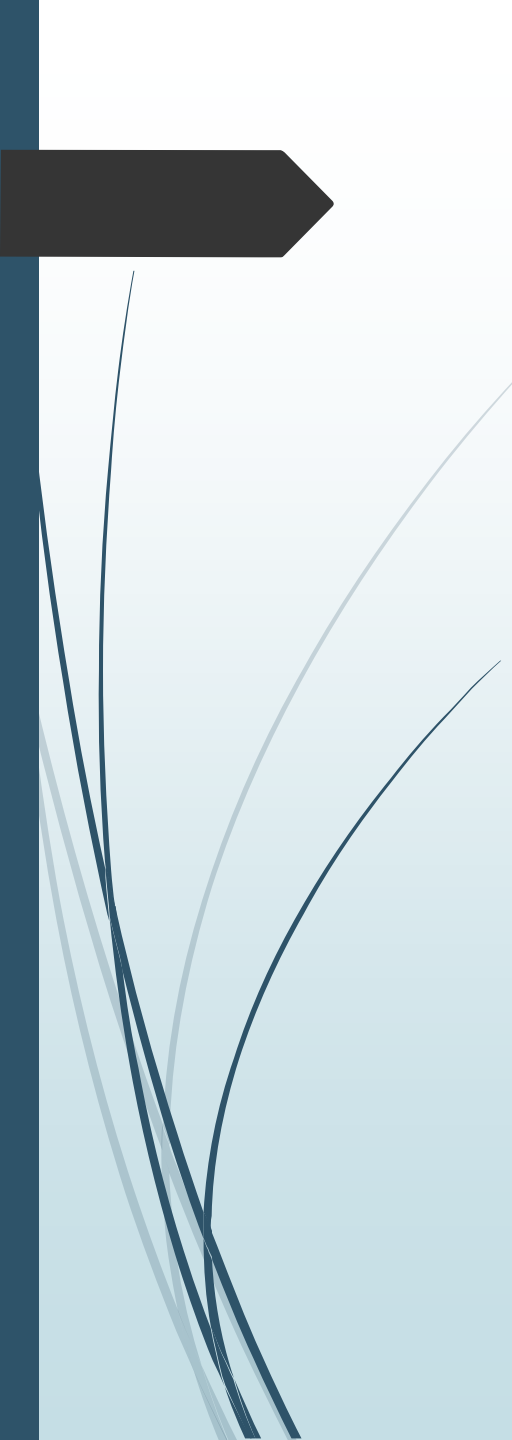
Linguaggio Python - Comparazione con Java

Caratteristica	Python	Java
Classi	<pre>Esempio: class Animal (): """ A simple class """ def __init__(self , name): self.name = name def say(self): print ("I am", self.name) class Dog(Animal): """ A simple subclass """ def say(self): print ("I am", self.name , \ "and I can bark !") d = Dog('Pippo') d.say ()</pre>	<pre>class Animal { private String name; public Animal(String name) { this.name = name ; } public say () { System.out.println("I am " + name); } } class Dog extends Animal { public Dog(String name){ super(name); } public say() { System.out.println("I am " + name + " and I can bark !"); } } static public void main(String[] args) { Dog d = new Dog(" Pippo"); d.say(); }</pre>

Linguaggio Python - Python 2 vs 3

Python 3.14 è l'ultima versione disponibile (3.15 in pre-release). **Non è compatibile** con le versioni 2.x. È stata introdotta per correggere alcuni ambiguità del linguaggio e per razionalizzare alcuni costrutti.

Cosa	Python 2	Python 3
print	istruzione	funzione
divisione intera	3/2=1	3/2=1.5
String	lista di caratteri ASCII	lista di caratteri UTF8!
certi valori di funzioni hanno cambiato tipo	dicts.keys(), dict.values() ritornano liste.	ritornano "viste" sullo stesso oggetto.
	map() e filter() ritornano liste	map() e filter() ritornano iteratori.
Operatori di confronto	(<, <=, >=, >) con tipi eterogenei ritorna false	(<, <=, >=, >) con tipi eterogenei solleva un'eccezione.
Sintassi	Alcune keyword sono state cambiate o eliminate!	
Molti altri cambiamenti più specifici!		



Documentazione

Documentazione da usare come principale riferimento:

<https://docs.python.org/3/>

Libro consigliato per chi vuole approfondire: **Mark Lutz. Learning Python.**

<https://learning-python.com/about-lp5e.html>

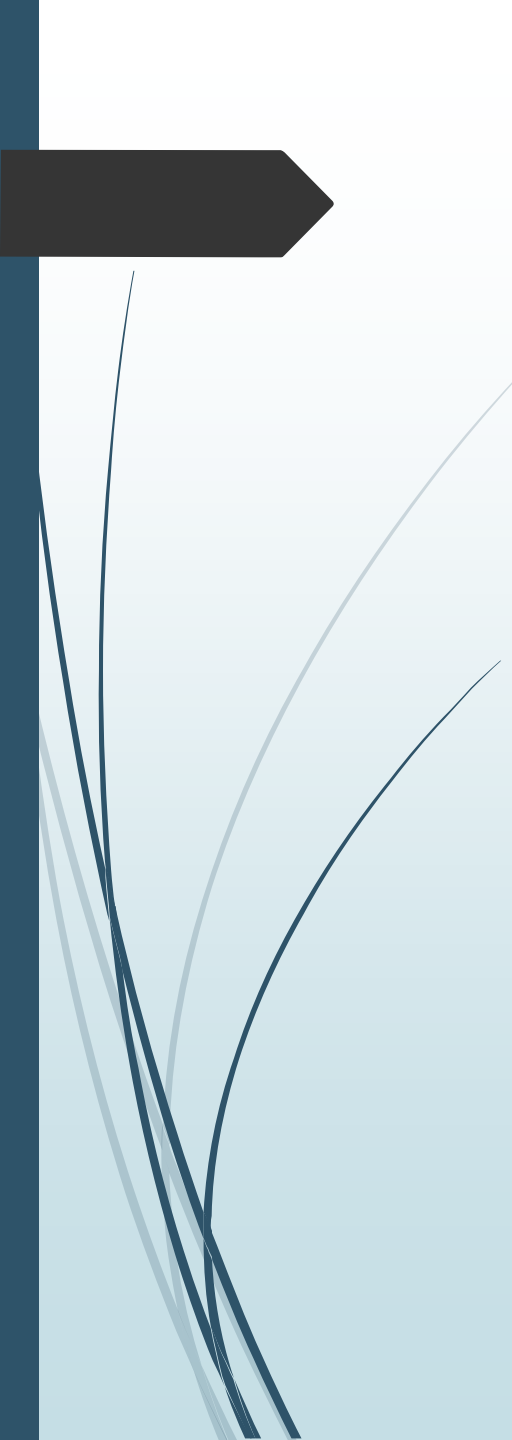
Sito di tutorial specifici su caratteristiche del linguaggio:

<https://pythonspot.com/en/all-tutorials/>



Ambiente di lavoro

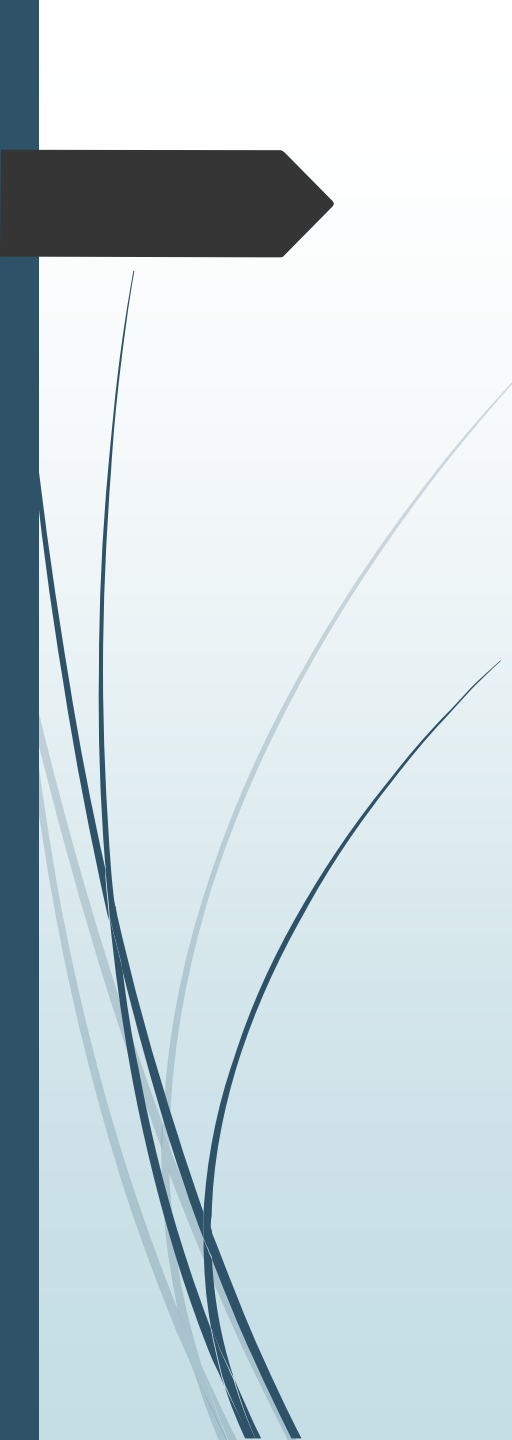
- Un'installazione standard di Python 3 (<https://www.python.org/>) rende disponibile un'interprete, `python`, che è un'interprete da console molto simile a `psql`.
- In molte distribuzioni di Linux e in Mac OS X, Python 2 è già presente. Talvolta anche Python 3. **Installare Python 3 se necessario!**
- Quando Python 3 è installato, è presente l'interprete `python3`:
 - `python` potrebbe essere collegato a Python 2. Usare `python3` per essere certi di usare Python 3.



Linguaggio Python - Ambiente di lavoro

Python in laboratorio Delta

```
$ python3
Python 3.6.4 ( default , Mar 9 2018 , 23:15:03)
[GCC 4.2.1 Compatible Apple LLVM 9.0.0 (clang -900.0.39.2)]
Type "help", "copyright", "credits" or "license" for more information.
>>> print ("Hello world!")
Hello world!
>>> exit ()
$
```



Linguaggio Python - Ambiente di lavoro

- Analogamente a quanto fatto per `psql`, dopo i primi esperimenti, conviene scrivere il proprio programma Python in un file ed eseguire il file.
- I programmi python hanno suffisso `.py`.

`python3 <filePython>` interpreta il file `filePython` come programma Python 3 e termina.

`python3 -i <filePython>` interpreta il file `filePython` come programma Python 3 e rimane aperto.

- Con Python 3, un file di programma deve essere scritto usando UTF8.

Python – Esempio Fibonacci

File fibonacci.py

```
# Serie di Fibonacci
a, b = 0, 1 # assegnamento in sequenza
n = int( input ("Fino a quale intero calcolare la serie ?"))
while b < n:
    __print (b, end='')
    __a, b = b, a+b
print()
```

Esecuzione di fibonacci.py

```
$ python3 fibonacci.py
Fino a quale intero calcolare la serie ? 100
1 1 2 3 5 8 13 21 34 55 89
$
```

Es. 1 – Scrittura di una tabella in un file

1 / 7

File scritturaDict.py:

```
"""
```

Si vuole salvare in un file una tabella delle spese definita come

```
table spese {  
    data DATE ,  
    voce VARCHAR ,  
    importo Decimal
```

```
}
```

In questo modulo si rappresenta la tabella come
una lista di record (**dict**) e si usa il formato CSV per salvare .

```
"""
```

```
#
```

```
#####
```

```
# IMPORT
```

```
import csv
```

```
import re
```

Es. 1 – Scrittura di una tabella in un file

2/7

File scritturaDict.py:

```
# CLASSI o funzioni di servizio
pattern = re.compile ("^\d {2 ,2}/\ d {2 ,2}/\ d{4 ,4}$")

def creaDict( data , voce , importo ):
    """ Ritorna un dict formato con gli argomenti .
    Formato: {'data': <data>, 'voce': <voce>, 'importo': <importo >}
    Fa il check sul formato data.
    """
    if not isinstance (data , str) or not pattern.match ( data ):
        print( "Data non è nel formato dd/mm/ aaaa" )
        exit()
    return { 'data': data , 'voce': voce, 'importo': float( importo ) }
```

Es. 1 – Scrittura di una tabella in un file

3/7

File scritturaDict.py:

```
# Creo la tabella e aggiungo dati
tabella = list ()
tabella.append( creaDict ( "24/02/2016", "Stipendio", 0.1) )
tabella.append( creaDict ( "24/02/2016", 'Stipendio "Bis"', 0.1) )
tabella.append( creaDict ( "24/02/2016", 'Stipendio "Tris"', 0.1) )
tabella.append( creaDict ( "27/02/2016", "Affitto", -0.3))

# Stampo la tabella da memoria
print( '=' * 50 )
print( " | {:10 s} | {: <20} | {: >10s} |".format( "Data", "Voce", "Importo" ) )
print( '-' * 50 )
for riga in tabella :
    print( " | {:10 s} | {: <20} | {: >10.2 f} |".format ( riga ['data'], riga ['voce'], riga
['importo']))
    print ( '=' * 50 )

# Calcolo il totale degli importi
tot = 0.0
for riga in tabella:
    tot += riga ['importo']
print( "La somma è {:.20 f}".format (tot) )
```

Es. 1 – Scrittura di una tabella in un file

4/7

File scritturaDict.py:

Salvo la tabella in un file in formato CSV

```
nomeFile = 'tabellaSpesa.csv'
```

```
with open( nomeFile, mode = 'w', encoding = 'utf -8 ') as csvFile:
```

```
    nomiCampi = ['data', 'voce', 'importo']
```

```
    writer = csv.DictWriter( csvFile , fieldnames = nomiCampi )
```

```
    writer.writeheader ()
```

```
    for riga in tabella :
```

```
        writer.writerow( riga )
```

Leggo dal file la tabella e la pongo in una nuova variabile

```
tab1 = list()
```

```
with open( nomeFile , mode = 'r', encoding = 'utf -8 ') as csvFile :
```

```
    reader = csv.DictReader( csvFile )
```

```
    for row in reader :
```

```
        tab1.append ( creaDict (row['data'], row['voce'], row['importo']))
```

Es. 1 – Scrittura di una tabella in un file

5/7

File scritturaDict.py:

```
# Calcolo il totale sulla nuova tabella
tot1 = 0
for riga in tab1 :
    tot1 += riga ['importo']
if tot == tot1:
    print( "I due totali sono uguali !« )
else :
    print( " Ops ... la tabella letta non ha gli stessi dati !" )
if tot == 0:
    print( " Eureka !" )
else :
    print( " Ops ... il totale non è corretto perchè non è 0!" )
```

Es. – Scrittura di una tabella in un file

6/7

File tabellaSpesa.csv

```
data ,voce , importo  
24/02/2016, Stipendio, 0.1  
24/02/2016, "Stipendio ""Bis""", 0.1  
24/02/2016, "Stipendio ""Tris""", 0.1  
27/02/2016, Affitto , -0.3
```

Es. 1 – Scrittura di una tabella in un file

7/ 7

Output di una esecuzione

```
=====
| Data      | Voce           | Importo |
=====
| 24/02/2016 | Stipendio      | 0.10    |
| 24/02/2016 | Stipendio "Bis" | 0.10    |
| 24/02/2016 | Stipendio "Tris" | 0.10    |
| 27/02/2016 | Affitto        | -0.30    |
=====
```

La somma è 0.000000000000000005551

I due totali sono uguali !

Ops ... il totale non è corretto perché non è 0!

Es. 2 – Scrittura di una tabella in un file

1/12

Il file scritturaClasse.py rappresenta un'evoluzione del file 'scritturaDict.py' in cui si risolve l'errore sugli importi e si dimostra un uso più avanzato di JSON:

File scritturaClasse.py:

```
"""
```

```
Si vuole salvare in un file una tabella delle spese definita come
```

```
table spese {
```

```
  data DATE ,
```

```
  voce VARCHAR ,
```

```
  importo Decimal
```

```
}
```

```
In questo modulo si rappresenta la tabella come un oggetto di una classe e si usa JSON per salvare .
```

```
Inoltre gli importi sono rappresentati come Decimal .
```

```
"""
```

Es. 2 – Scrittura di una tabella in un file

2/12

File scritturaClasse.py:

```
# IMPORT
import datetime
import decimal
import json
import re
#
#####
#
# CLASSI o funzioni di servizio
pattern = re.compile( "^\\d {2 ,2}/\\ d {2 ,2}/\\ d{4 ,4}$" )
def creaDict( data, voce, importo ):
    """
    Ritorna un dict formato con gli argomenti .
    Formato : { ' data '<data >,' voce '<voce >,' importo :< importo > }.
    Fa il check sul formato data e su importo che deve essere Decimal .
    """
```

Es. 2 – Scrittura di una tabella in un file

3/12

File scritturaClasse.py:

```
__if not isinstance( data, str ) or not pattern.match( data ):
__ __print( "Data non è nel formato dd/mm/aaaa" )
__ __exit()
__if not isinstance( importo , decimal.Decimal ) and not isinstance ( importo, str ):
__ __print ( "Importo deve essere un Decimal o una stringa che rappresenta un importo."
)
__ __exit()
__return{ 'data': data, 'voce': voce, 'importo': decimal.Decimal ( importo ) }
```

Ora 'importo' è forzato ad essere di tipo Decimal, che ha proprietà simili Decimal di SQL

Es. 2 – Scrittura di una tabella in un file

4/12

File scritturaClasse.py:

```
class Spese :
```

```
    """
```

L'attributo 'tabella' rappresenta le spese organizzate come dict(primaryKey, riga).

La primaryKey è data dalla coppia (data, voce).

La riga è un dict creato dal metodo creaDict().

L'attributo 'ultimaModifica' rappresenta il timestamp dell 'ultima modifica .

```
    """
```

```
def makeKey (data , voce ):
```

```
    return data + "_%_" + voce
```

```
def __init__ (self, inputTab = {} , istante = datetime.datetime.now()):
```

```
    self.tabella = dict ( inputTab )
```

```
    self.ultimaModifica = istante
```

Es. 2 – Scrittura di una tabella in un file

5/12

File scritturaClasse.py:

```
def add( self, data, voce, importo ):  
    self.tabella[Spese.makeKey( data , voce )] = creaDict( data , voce , importo )  
    self.ultimaModifica = datetime.datetime.now()  
    return importo  
  
def remove( self, data, voce ):  
    del self.tabella[Spese.makeKey( data , voce )]  
    self.ultimaModifica = datetime.datetime.now()  
  
def get( self , data , voce ):  
    return self.tabella[self.makeKey( data , voce )]  
  
def items ( self ):  
    return self.tabella.items()
```

Es. 2 – Scrittura di una tabella in un file

6/12

File scritturaClasse.py:

```
#  
#####  
# Creo la tabella e aggiungo dati  
tab = Spese ()  
tab.add( "24/02/2016 ", " Stipendio ", "0.1")  
tab.add( "24/02/2016 ", 'Stipendio "Bis"', "0.1" )  
tab.add( "24/02/2016 ", 'Stipendio "Tris"', "0.1" )  
tab.add( "27/02/2016 ", " Affitto ", " -0.3" )  
# Stampo la tabella da memoria  
print( '=' * 50 )  
print( " | {:10 s} | {: <20} | {: >10s} | ".format( "Data", "Voce", "Importo" ) )  
print( '-' * 50 )  
for riga in tab.tabella.values():  
    print( " | {:10 s} | {: <20} | {: >10.2 f} | ".format( riga['data'], riga['voce'], riga['importo']))  
    print( '=' * 50 )
```

Es. 2 – Scrittura di una tabella in un file

7/12

File scritturaClasse.py:

```
# Calcolo il totale degli importi
tot = decimal.Decimal(0)
for riga in tab.tabella.values():
    tot += riga['importo']
print ("La somma è {:.20 f}".format (tot))
```

Es. 2 – Scrittura di una tabella in un file

8/12

File scritturaClasse.py:

```
# Salvataggio in un file
class MyEncoder( json.JSONEncoder ):
    """ Codifica gli oggetti di tipo Decimal , Date o Spese . """
    def default( self, o ):
        if isinstance( o, decimal.Decimal ):
            return float(o)
        if isinstance( o, datetime.datetime ):
            return o.isoformat()
        if isinstance( o, Spese ):
            return{ "tabella": o.tabella , "ultimaModifica": o.ultimaModifica }
        return json.JSONEncoder.default( self, o )

# Salvo la tabella in un file
nomeFile = 'database.json '
with open( nomeFile, mode ='w', encoding ='utf -8 ' ) as file :
    json.dump( tab, file, cls=MyEncoder, indent =4 )
```

Es. 2 – Scrittura di una tabella in un file

9/12

File scritturaClasse.py:

```
def myDecoder ( jsonObj ):  
    """ Decodifica oggetti di tipo Spesa . """  
    if 'tabella' in jsonObj :  
        tab = jsonObj['tabella']  
        istante = datetime.datetime.strptime( jsonObj['ultimaModifica'], "%Y -%m -%dT%H:%M:%S.%f" )  
        return Spese ( tab , istante )  
    else :  
        return jsonObj  
  
# Leggo dal file la tabella e la pongo in una nuova variabile  
with open( nomeFile , mode ='r', encoding ='utf -8' ) as file :  
    tab1 = json.load( file , object_hook = myDecoder, parse_float = decimal.Decimal )
```

Es. 2 – Scrittura di una tabella in un file

10/12

File scritturaClasse.py:

```
# Calcolo il totale sulla nuova tabella
tot1 = decimal.Decimal(0)
for riga in tab1.tabella.values():
    tot1 += riga ['importo']
if tot == tot1 :
    print( "I due totali sono uguali !" )
if tot == 0:
    print( " Eureka !" )
else :
    print( "Ops ... il totale non è corretto !" )
```

Es. 2 – Scrittura di una tabella in un file

11/12

File database.json (compattato)

```
{
  "tabella": {
    "24/02/2016_%_StipendioTris":
      { "importo": 0.1, "data": "24/02/2016", "voce": "S... Tris« },
    "27/02/2016_%_Affitto":
      { "importo": -0.3, "data": "27/02/2016", "voce": "Affitto" },
    "24/02/2016_%_StipendioBis":
      { "importo": 0.1, "data": "24/02/2016", "voce": "Sti ... Bis"},
    "24/02/2016_%_Stipendio":
      { "importo": 0.1, "data": "24/02/2016", "voce": " Stipendio" }
  },
  "ultimaModifica": "2016 -04 -15 T12 :06:50.899597"
}
```

Es. 2 – Scrittura di una tabella in un file 12/12

Output

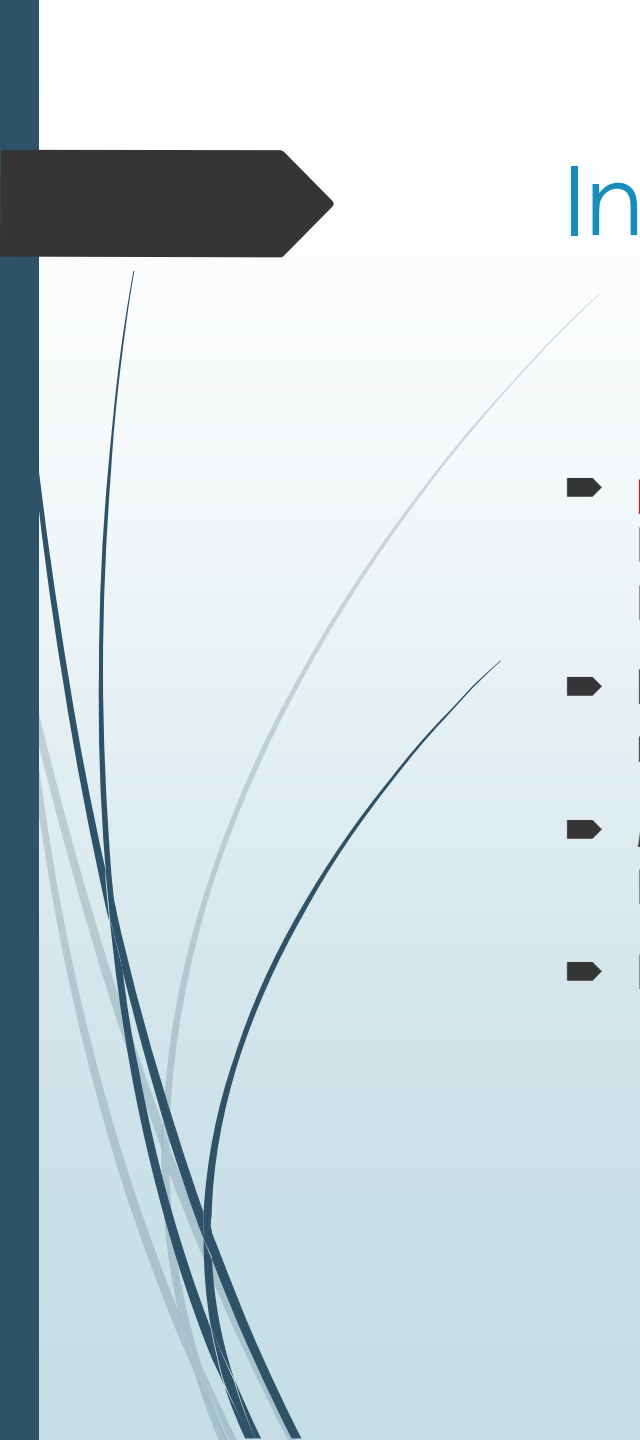
```
=====
| Data      | Voce          | Importo |
-----
| 27/02/2016 | Affitto      | -0.30 |
| 24/02/2016 | Stipendio "Tris" | 0.10 |
| 24/02/2016 | Stipendio     | 0.10 |
| 24/02/2016 | Stipendio "Bis" | 0.10 |
=====
```

La somma è 0.000000000000000000000000
I due totali sono uguali !
Eureka !



Introduzione a psycopg2

- **DB-API v2.0** è la Application Program Interface (API) ufficiale (Python Enhancement Proposals (PEP) 249
- <https://www.python.org/dev/peps/pep-0249>) che descrive come un modulo Python deve accedere a una base di dati esterna.
- Diversi gruppi di sviluppo hanno reso disponibili moduli (librerie) DB-API per diversi tipi di DBMS.
- Alla pagina <https://wiki.python.org/moin/DatabaseInterfaces> c'è l'elenco aggiornato dei moduli disponibili.
- Per PostgreSQL ci sono più di 10 (2022-05-26) implementazioni diverse di DB-API v2.0.
- In questa lezione si introduce DB-API v2.0 considerando l'implementazione **psycopg2** (<http://initd.org/psycopg/>).



Introduzione a psycopg2

- **psycopg2** è una libreria scritta quasi completamente in C e implementa DB-API v2.0 mascherando la libreria C ufficiale libpq del gruppo di PostgreSQL (<https://www.postgresql.org/docs/current/libpq.html>)
- Implementa i cursori lato client e lato server, comunicazione asincrona, le notifiche e il comando COPY
- Molti tipi di dati Python sono supportati e mappati nei tipi di dati PostgreSQL.
- La mappatura dei tipi di dati può essere personalizzata in modo semplice.

Fondamenti di DB-API v2.0: Connection

L'accesso a un database avviene tramite un oggetto di tipo **Connection**. Il metodo **connect(...)** accetta i parametri necessari per la connessione e ritorna un oggetto **Connection**.

```
connector = psycopg2.connect( host="dbserver.scienze.univr.it", database="db0", \  
user="user0", password="xxx" )
```



Fondamenti di DB-API v2.0: Connection

Classe **Connection** – metodi principali:

1. **cursor()**: ritorna un cursore della base di dati. Un oggetto cursore permette di inviare comandi SQL al DBMS e di accedere al risultato del comando restituito dal DBMS.
2. **commit()**: registra la transazione corrente. Attenzione! Normalmente una connessione apre una transazione al primo invio di comandi. Se non si esegue un **commit()** prima di chiudere, tutte le eventuali modifiche/inserimenti vengono persi.
3. **rollback()**: abortisce la transazione corrente.
4. **close()**: chiude la connessione corrente. Implica un **rollback()** automatico delle operazioni non registrate.

Fondamenti di DB-API v2.0: Connection

5. **autocommit**: proprietà r/w. Se **True**, ogni comando inviato è una transazione isolata. Se **False** (default) il primo comando inviato inizia una transazione, che deve essere chiusa con **commit()** o **rollback()**.
6. **readonly**: proprietà r/w. Se **True**, nella sessione non si possono inviare comandi di modifica dati. Il default è **False**.
7. **isolation_level**: proprietà r/w. Modifica il livello di isolamento per la prossima transazione. Valori leciti: 'READ UNCOMMITTED', 'READ COMMITTED', 'REPEATABLE READ', 'SERIALIZABLE', 'DEFAULT'. Meglio assegnare questa variabile subito dopo la creazione della connessione.

```
connector = psycopg2.connect(...)  
connector.isolation_level = 'REPEATABLE READ'
```

Fondamenti di DB-API v2.0: Cursore

Un **cursore** gestisce l'interazione con la base di dati: mediante un cursore è possibile inviare una comando SQL e accedere all'esito e ai dati di risposta del comando.

Classe **Cursor** – metodi principali:

1. **execute(comando, parametri)**: prepara ed esegue un 'comando' SQL usando i 'parametri'. Parametri devono essere passati come tupla o come dict. Il comando ritorna None. Eventuali risultati di query si devono recuperare con il **fetch*()**.

```
cur.execute( "CREATE TABLE test( id SERIAL PRIMARY KEY, num integer, data  
varchar)" )  
cur.execute( "INSERT INTO test (num, data) VALUES (%s, %s)", (100, "abc'def") )  
# psycopg2 fa le conversioni!
```

Fondamenti di DB-API v2.0: cursore

2. **executemany(comando, parametri)**: prepara ed esegue un 'comando' SQL per ciascun valore presente nella lista 'parametri'.

```
cur.executemany( "INSERT INTO test (num, data ) VALUES (%s, %s)",  
                [(100, "abc'def"), (None , 'dada'), (42 , 'bar')]  
                )
```

NOTA

Per come è attualmente implementato, **executemany()** è meno efficiente di un ciclo for con **execute()** o, meglio ancora, di un unico insert con più tuple:

```
cur.execute( "INSERT INTO test (num, data) VALUES (%s, %s), (%s, %s), (%s,  
%s)",  
            (100, "abc'def", None, 'dada', 42, 'bar'))
```

Fondamenti di DB-API v2.0: Cursore

`cursor.execute*` accetta `%s` come indicatore di posizione parametro. La conversione dal tipo Python al dominio SQL è automatica per tutti i tipi fondamentali.

```
cur.execute("INSERT INTO test1 (id , date_val , item ) VALUES \  
(%s, %s, %s)", (42 , datetime.date(2005 , 11, 18), "O'Reilly" ))
```

è convertita in SQL

```
INSERT INTO test1 ( id , date_val , item ) VALUES ( 42 , '2005 -11 -18 ' , 'O"Reilly'  
);
```

Fondamenti di DB-API v2.0: Cursore

Tipo Python	Tipo PostgreSQL	Note
None	NULL	Rappresenta l'assenza di valore.
bool	BOOL	Vero/Falso.
float	REAL o DOUBLE	Numeri con virgola (approssimati).
int	SMALLINT, INTEGER, bigint	Numeri interi di varie dimensioni.

Maggiori dettagli: <http://initd.org/psycopg/docs/usage.html>

Fondamenti di DB-API v2.0: Cursore

Decimal	NUMERIC	Fondamentale: garantisce la precisione esatta per i calcoli finanziari.
str	VARCHAR o TEXT	Stringhe di testo.
date	DATE	Solo la data (giorno/mese/anno).
time	TIME o timez	Solo l'orario.
datetime	TIMESTAMP o timestampz	Data e ora complete.



Fondamenti di DB-API v2.0: cursore

Tipico errore da principiante: SQL Injection

Alcuni pensano sia più efficiente e facile scrivere

```
cur.execute("SELECT 1 FROM users WHERE name='" + user + "' AND pw='" + pw + "'")
```

Anziché

```
cur.execute("SELECT 1 FROM users WHERE name=%s AND pw=%s", (user, pw))
```

dove user e pw sono letti da input.

Altri, più intelligenti, possono sfruttare questa ingenuità per fare altro:

Fondamenti di DB-API v2.0: cursore

Tipico errore: SQL Injection

Assegnando:

- ' OR TRUE -- a user
- un qualsiasi carattere a pw

il risultato è

```
SELECT 1 FROM users WHERE name=" OR TRUE --" AND pw='a'
```

Fondamenti di DB-API v2.0: cursore

Tipico errore: SQL Injection

```
cur.execute("SELECT 1 FROM users WHERE name='" + user + "' AND pw='" + pw + "'")
```

Assegnando:

- `"; DROP TABLE users CASCADE; --` a user
- un qualsiasi carattere a pw

il risultato è

```
SELECT 1 FROM users WHERE name="";  
DROP TABLE users CASCADE; --' AND pw='a'
```

Fondamenti di DB-API v2.0: Cursore

3. `fetchone()`: ritorna una tupla della tabella risultato. Si può usare dopo un `execute("SELECT ...")`. Se non ci sono tuple, ritorna `None`.

```
>>> cur.execute( "SELECT * FROM test WHERE id = %s", (3, ) )  
>>> cur.fetchone()  
(3, 42, 'bar')
```

Fondamenti di DB-API v2.0: Cursore

4. **fetchmany**(<numero>) : ritorna una lista di tuple della tabella risultato di lunghezza max <numero>. Si può usare dopo un `execute("SELECT ...")`. Se non ci sono tuple, ritorna una lista vuota.

```
>>> cur.execute( "SELECT * FROM test WHERE id < %s", (4 ,) )
>>> cur.fetchmany(3)
[(1 , 100 , " abc 'def "), (2, None , 'dada '), (3, 42, 'bar ')]
>>> cur.fetchmany(2)
[]
```

Fondamenti di DB-API v2.0: Cursore

5. Dopo un `execute("SELECT ...")`, il cursore è un iterabile sulla tabella risultato. È possibile quindi accedere alle tuple del risultato anche con un ciclo.

```
>>> cur.execute( "SELECT * FROM test WHERE id < %s", (4, ) )
>>> for record in cur:
...     print (record, end=", ")
(1, 100 , " abc 'def ") , (2, None , 'dada ') , (3, 42, 'bar ') ,
```

Fondamenti di DB-API v2.0: cursore

6. **rowcount**: di sola lettura, = numero di righe prodotte da ultimo comando. - 1 indica che non è possibile determinare il valore.
7. **statusmessage**: di sola lettura, = messaggio ritornato dall'ultimo comando eseguito.

```
>>> cur.execute("INSERT INTO test (num , data ) VALUES (%s, %s)", (42 ,  
'bar'))  
>>> cur.statusmessage  
'INSERT 0 1'
```

Schema per usare psycopg2

- psycopg2 non fa parte della distribuzione standard.
- Python3 ha un meccanismo di installazione semplice (pip) dei moduli che sono registrati presso Python Packaging Index.
- pip può installare a livello di sistema (richiede diritti amministratore) o a livello utente. Dettagli: <https://docs.python.org/3/installing>.
- Installazione Psycopg2 su Ubuntu Desktop:
 1. Si assume che Python3 sia già installato come pure il programma pip3. (Dalla versione Python 3.5, pip è già presente nella distribuzione).
 2. Da una shell, si avvia l'installazione di Psycopg2 con il comando: `pip3 install --user psycopg2 psycopg2 - binary`
 3. Dopo qualche compilazione, il modulo viene salvato in `~/.local/lib/python3.x/site-packages/` ed è disponibile automaticamente in ogni sessione Python3.



Schema per usare Psycopg2

Lo schema tipico di un modulo Python che comunica con un DBMS PostgreSQL via Psycopg2 deve:

1. aprire una connessione tramite `conn = psycopg2.connect(...)`,
2. eventualmente modificare le proprietà di livello di isolamento, autocommit, readonly.
3. creare un cursore tramite `cur = conn.cursor()`,
4. eseguire le operazioni previste,
5. se la sessione non è in autocommit, eseguire un `conn.commit()` se si sono dati comandi SQL di aggiornamento (registra le modifiche) (o `conn.rollback()` per annullare),
6. chiudere il cursore, `cur.close()`, e la connessione, `conn.close()`.



Schema per usare Psycpg2

- Dalla versione 2.5 della libreria, la gestione dei close e dei commit è semplificata se si usa il costrutto **with**:
- Quando si usa una **connessione** con il **with**, all'uscita del blocco viene fatto un commit automatico e la connessione non viene chiusa.
- Quando si usa/crea un **cursore** con il **with**, all'uscita del blocco viene fatto un close automatico del cursore.

Schema per usare Psycopg2

1. `conn = psycopg2.connect (...)`
2. `with conn:`
3. `__with conn.cursor () as cur1:`
4. `__ __...`
5. `__print (" Qui cur1 è stato chiuso ")`
6. `print ( "Qui è stato fatto solo un commit : conn è ancora aperta!")`
7. `with conn :`
8. `__with conn . cursor () as cur2 :`
9. `__ __...`
10. `__print (" Qui cur2 è stato chiuso ")`
11. `print( "Qui è stato fatto un commit: conn è ancora aperta!" )`
12. `conn.close()`



Connessioni e Cursori

- Si deve porre attenzione alla combinazione di cursori sulla medesima connessione
- Aprire una connessione costa in tempo (e spazio). Meglio aprire/chiudere poche connessioni in un'esecuzione.
- Con un oggetto connessione si possono creare più cursori. Questi cursori **condividono** la connessione.
- Psycopg2 garantisce solo che le istruzioni inviate dai cursori vengono sequenzializzate.
- Quindi non si possono gestire transazioni concorrenti usando diversi cursori sulla medesima connessione.
- Regola pratica: usare più cursori sulla medesima connessione quando si fanno transazioni in auto-commit o solo transazioni di sola lettura.

Schema per usare Psycopg2

Esempio di modulo basato su Psycopg2 (1/5)

''''''

Gestione semplice tabella Spese su PostgreSQL .
Manca la gestione delle eccezioni !

''''''

```
from datetime import date  
from decimal import Decimal
```

```
import psycopg2
```

Il seguente import definisce delle variabili con dati sensibili come le
password. \

Metodo semplice per isolare in un solo file tali parametri

```
from myAppConfig import myHost, myDatabase, myUser, myPas
```

Schema per usare Psycopg2

Esempio di modulo basato su Psycopg2 (2/5)

```
connessione = psycopg2.connect ( host=myHost , \  
database=myDatabase, user=myUser, password=myPasswd )  
with connessione :  
    __with connessione.cursor () as cursore :  
        __cursore.execute ( \  
            __ __ """"CREATE TABLE IF NOT EXISTS Spese ( \  
            __ __ __id SERIAL PRIMARY KEY, \  
            __ __ __data DATE NOT NULL, \  
            __ __ __voce VARCHAR NOT NULL, \  
            __ __ __importo NUMERIC NOT NULL \  
            __ __ __)"""" \  
        __ __ )  
print( 'Esito della creazione della tabella Spese : {s}\n Eventuali \  
notifiche : \  
{s}'.format( cursore.statusmessage , connessione.notices [ -1]))
```

Schema per usare Psycopg2

Esempio di modulo basato su Psycopg2 (3/5)

```
__ __ cursore.execute( """SELECT count (*) FROM Spese""" )
__ __ numeroRighe = cursore.fetchone()[0]
__ __ if numeroRighe == 0:
__ __ __ cursore.execute( """INSERT INTO \
Spese(data, voce, importo ) VALUES (%s, %s, %s),
__ __ __ __ (%s, %s, %s),
__ __ __ __ (%s, %s, %s),
__ __ __ __ (%s, %s, %s)""" ,
__ __ __ __ ( date(2016, 2, 24), "Stipendio", Decimal( "0.1" ),
__ __ __ __ date(2016, 2, 24), "Stipendio 'Bis'", Decimal( "0.1" ),
__ __ __ __ date(2016, 2, 24), "Stipendio 'Tris'", Decimal( "0.1" ),
__ __ __ __ date (2016 ,2 ,27) , " Affitto ", Decimal ( " -0.3" ))
__ __ __ __ )
__ __ __ print ("Esito dell' inserimento delle 4 tuple : \
{:s}".format( cursore.statusmessage ))
```

Schema per usare Psycpg2

Esempio di modulo basato su Psycpg2 (4/5)

```
__ __else :
__ __print ( "La tabella è già presente con delle tuple e \
quindi nessuna tupla è stata aggiunta." )
__connessione.commit () # Tutti i dati sono salvati

__with connessione.cursor () as lettore:
__ __lettore.execute( """SELECT id, data, voce, importo \
FROM Spese""" )
__ __print( "Esito della selezione di tutte le tuple : \
{:s}".format( cursore.statusmessage ) )
__ __print( '=' * 55 )
__ __patternRiga = " | {: >2s} | {:10 s} | {: <20s} | {: >10s} | "
```

Schema per usare Psycopg2

Esempio di modulo basato su Psycopg2 (5/5)

```
__ __print( patternRiga.format( "N", "Data", "Voce", "Importo" ))
__ __print ( '-' * 55)
__ __tot = Decimal ("0")
__ __patternRiga = " | {: >2d} | {:10 s} | {: <20s} | {: >10.2 f} | "
__ __for tupla in lettore:
__ __ __print( patternRiga.format( tupla[0], tupla[1].isoformat(),
tupla[2], tupla[3] ))
__ __ __tot += tupla [3]

__ __print( '-' * 55 ) # A questo punto la tabella è stampata in output
connessione.close()
print( "{: >40s} {:10.2 f}".format( "Totale", tot ))
```

Schema per usare Psycopg2

Esempio di modulo basato su Psycopg2 - Esecuzione

Esito di una esecuzione:

Esito creazione tabella : CREATE TABLE

Esito inserimento tabella : INSERT 0 4

```
=====
| N | Data      | Voce          | Importo |
-----
| 1 | 2016 -02 -24 | Stipendio      | 0.10 |
| 2 | 2016 -02 -24 | Stipendio "Bis" | 0.10 |
| 3 | 2016 -02 -24 | Stipendio "Tris" | 0.10 |
| 4 | 2016 -02 -27 | Affitto        | -0.30 |
-----
```

Totale 0.00